ADC Driver Development and Application

Melanie Picen – 35032, Sebastian Lizalde-Student 38241, Michelle Urbina-37964
CETYS Universidad

***Abstract*—This file implements the Analog Sensor module for the STM32F411RE microcontroller. It provides three functions to interface a potentiometer connected to GPIO pin PA0 (ADC1 Channel 0): initialization, conversion trigger, and averaged value reading. The module relies on the ADC driver layer to perform low-level register operations, acquiring ten consecutive samples with a 5 ms interval between readings to eliminate signal noise, and returning a 12-bit averaged result in the range of 0 to 4095. The potentiometer output is mapped to a PWM duty cycle to control LED brightness in real time.**

Keywords—STM32F411RE, ADC, Analog-to-Digital Converter, GPIO, PWM, Pulse Width Modulation, potentiometer, sensor module, hardware abstraction, layered architecture, embedded systems, duty cycle, TIM4, real-time control.

I. Introduction

This report presents the design of an ADC-Based Analog Input Control system that incorporates the STM32F411RE microcontroller, referred to here as the STM32. The objective of this Lab Assignment is to implement an ADC driver following the established layered architecture, separating the system into three distinct layers: the Application Layer, the Driver Layer, and the Hardware Layer. This architecture allows each module to operate independently, improving code reusability and maintainability across lab assignments.

The system implements two primary modules: an Analog Sensor module that reads input from a potentiometer connected to pin PA0 using ADC1 Channel 0, and a PWM module reused from Lab Assignment 02 that controls LED brightness on pin PB6 using TIM4. The ADC driver converts the analog voltage at PA0 into a 12-bit digital value between 0 and 4095, which is then mapped to a PWM duty cycle between 0% and 100%. This integration enables real-time analog input control: as the potentiometer is rotated, the LED brightness adjusts accordingly and continuously.

The materials used for this Lab Assignment include the STM32F411RE Nucleo-64 development board, a potentiometer connected to the analog input pin PA0, Visual Studio Code with Git Bash as the development environment, and the laboratory hardware for functional verification.

II. Process Description

System Overview

The system implements a potentiometer-controlled PWM application on the STM32F411RE Nucleo-64 development board, following the same layered driver architecture established in previous lab assignments. The system is composed of four interdependent modules: the GPIO Driver, the TIM Driver, the ADC Driver, and two application-level modules — the Analog Sensor module and the PWM module. The ADC Driver and Sensor module were developed for this assignment, while the GPIO Driver, TIM Driver, and PWM module were reused directly from Lab Assignment 02. The application layer reads an analog voltage from a potentiometer connected to pin PA0, converts it to a 12-bit digital value using ADC1, maps the result to a PWM duty cycle, and drives an LED on pin PB6 with variable brightness in real time.

ADC Driver Initialization

System execution begins with a global hardware initialization sequence. The gpio_init() function enables the RCC clocks for all GPIO ports on the AHB1 bus. The ADC driver is then initialized through adc_init(), which performs the complete ADC1 peripheral configuration. It enables the ADC1 clock through the RCC_APB2ENR register by setting the ADC1EN bit, configures the ADC common prescaler to divide the input clock by four through the CCR register, sets the resolution to 12 bits by clearing the RES field in CR1, disables scan mode, configures right-aligned data output by clearing the ALIGN bit in CR2, disables external trigger sources by clearing EXTEN and JEXTEN, and sets the regular sequence length to one conversion through SQR1. Following initialization, adc_enableAdc() sets the ADON bit in CR2 to power on the ADC peripheral and executes a stabilization delay of 300 countdown cycles to allow the internal voltage reference to settle before the first conversion is performed.

Table 1. ADC1 register configuration during initialization.

Register	Field	Value	Description
RCC_APB2ENR	ADC1EN	1	Enable ADC1 peripheral clock
ADC_CCR	ADCPRE	01	Prescaler divide by 4
CR1	RES	00	12-bit resolution
CR1	SCAN	0	Single channel mode
CR2	ALIGN	0	Right-aligned data
CR2	EXTEN	00	Software trigger only
SQR1	L	0000	One conversion

			in sequence
CR2	ADON	1	ADC Power On

ADC Channel Configuration

Once the ADC is initialized and enabled, the input channel is configured through adc_setChannel(). For this application, Channel 0 is selected, corresponding to GPIO pin PA0. The function writes the channel number into the SQR3 register, which defines the first conversion in the regular sequence. It then calls adc_setSampleTime() to configure the sampling time for Channel 0 in the SMPR2 register using the default value of ADC_SAMPLE_TIME_DEFAULT (84 cycles), which provides sufficient settling time for the potentiometer signal at the ADC clock frequency.

GPIO Analog Mode Configuration

Before the ADC can read the potentiometer voltage, pin PA0 must be configured in analog input mode. This is performed in sensor_init() through a two-step sequence. First, gpio_initPort(GPIO_PORT_A) enables the RCC clock for Port A by setting the corresponding bit in the AHB1ENR register.

Then gpio_setPinMode(GPIO_PORT_A, 0, GPIO_MODE_ANALOG) writes the analog mode value 0b11 into the MODER register bits corresponding to pin 0, disconnecting the pin from the digital logic and routing it directly to the ADC input multiplexer. Table 2 summarizes the GPIO configuration for PA0.

Table 2. GPIO configuration for PA0 analog input.

Register	Operation	Value	Description
RCC_AHB1ENR	Set GPIOAEN	1	Enable Port A clock
GPIOA_MODER	Bits [1:0]	11	Analog mode for pin 0

Sensor Module Operation

The Sensor module provides a three-function interface that abstracts all ADC operations from the application layer. Initialization is performed once at startup by sensor_init(), which sequentially calls gpio_initPort(), gpio_setPinMode(), adc_init(), adc_enableAdc(), adc_setChannel(), and timer_init() to prepare all required peripherals.

Conversion triggering is handled by sensor_startConversion(), which calls adc_startSingleConversion(). This function first clears the CONT bit in CR2 to ensure single conversion mode, then clears the EOC and STRT flags in the SR register, and finally sets the SWSTART bit in CR2 to initiate the conversion in software.

Value acquisition is handled by sensor_readValue(), which implements a ten-sample averaging algorithm. The function calls sensor_startConversion() ten times, reading the ADC result after each conversion through adc_readData(). The adc_readData() function polls the EOC flag in the SR register and blocks until it is set by hardware, confirming that the conversion is complete, then reads the 12-bit result from the DR register. A 5 ms delay is inserted between samples using timer_delay_ms(5) to allow the potentiometer signal to settle between readings. The ten samples are accumulated and divided to produce the averaged result.

Table 3. Sensor value acquisition sequence in sensor_readValue().

Iteration	Operation	Register	Result
1-10	sensor_startConversion()	CR2.SWSTART = 1	Conversion triggered
1-10	Poll EOC flag	SR.EOC	Wait for completion
1-10	Read result	DR [11:0]	12-bit sample added to sum
1-10	timer_delay_ms(5)	TIM2	5 ms settling delay
After loop	sum/10	-	Averaged 12-bit result

PWM Module Reuse and LED Brightness Control

The PWM module was reused without modification from Lab Assignment 02. It uses TIM4 Channel 1 configured in PWM Mode 1 on pin PB6 through Alternate Function 2. The led_init() function initializes the PWM output by calling pwm_init() and starting the signal with pwm_start(). LED brightness is controlled through led_setBrightness(), which was added to the LED module for this assignment. The function calls pwm_setSignal(1000, duty_cycle), updating the CCR1 register of TIM4 with a new compare value while keeping the frequency fixed at 1 kHz.

Application Integration and Real-Time Control

In main.c, the complete potentiometer-to-LED feedback loop is executed continuously after initialization. The led_init() function prepares the PWM output on PB6, and sensor_init() prepares the ADC input on PA0. The main loop reads the averaged potentiometer value through sensor_readValue(), maps it to a duty cycle percentage using the formula:

duty_cycle = (sensor_value × 100) / 4095

and applies it to the LED through led_setBrightness().

Because sensor_readValue() already averages ten samples with 5 ms intervals, each complete loop iteration takes approximately 50 ms, producing a refresh rate of roughly 20 updates per second — fast enough for smooth, continuous LED brightness response to potentiometer rotation.

Challenge: Circuit Not Responding After Successful Compilation

The primary challenge encountered during this lab assignment occurred after the code compiled successfully in Visual Studio Code but the LED did not respond when the circuit was powered. The code built without errors and was flashed to the STM32 correctly, but the LED remained off regardless of the potentiometer position.

The issue was diagnosed by systematically verifying each layer of the system independently. The hardware connections were reviewed first — confirming that the potentiometer center pin was connected to PA0, the LED anode was connected through a 220Ω resistor to PB6, and the cathode was connected to GND. After confirming the connections were correct, the software initialization sequence was reviewed and the missing pwm_start() call was identified. Without starting the TIM4 counter, the PWM signal was never generated regardless of the duty cycle value written to CCR1. Once pwm_start() was added to the initialization sequence, the system responded correctly and LED brightness varied smoothly with potentiometer rotation.

System Demostration

The complete system was verified on the STM32F411RE Nucleo-64 hardware. With the potentiometer rotated to its minimum position, the ADC read a value near 0, the duty cycle mapped to approximately 0%, and the LED remained off. With the potentiometer at its maximum position, the ADC read a value near 4095, the duty cycle mapped to approximately 100%, and the LED reached full brightness. Intermediate positions produced proportional brightness levels, confirming correct real-time sensor-to-PWM mapping. A video demonstration of the system operating on hardware is available at the link provided in the Results section.

III. Results

The lab assignment was successfully completed, achieving real-time potentiometer-controlled LED brightness on the STM32F411RE Nucleo-64 development board. All three tasks defined in the assignment were implemented and verified on hardware.

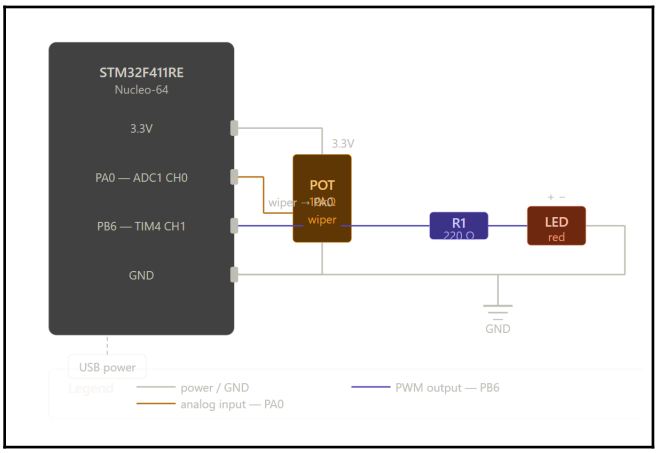
The potentiometer connected to PA0 produced stable and continuous ADC readings across the full 12-bit range of 0 to 4095. The ten-sample averaging algorithm in sensor_readValue() effectively eliminated signal noise, producing smooth brightness transitions without flickering as the potentiometer was rotated. The ADC values were correctly mapped to PWM duty cycle percentages and applied to the LED through TIM4 on pin PB6, demonstrating correct real-time sensor-to-actuator integration.

The observed hardware behavior is summarized in Table 4.

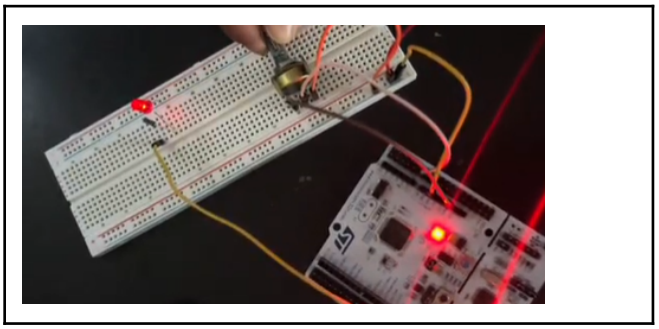
Table 4. Observed hardware behavior at key potentiometer positions.

Potentiometer Position	ADC Value (approx.)	Duty Cycle	LED Brightness
Minimum (fully left)	~0	~0%	Off
Quarter turn	~1024	~25%	Dim
Center	~2048	~50%	Medium
Three-quarter turn	~3072	~75%	Bright
Maximum (fully right)	~4095	~100%	Full brightness

Circuit Schematic



Circuit in Real Life



Video Demonstration

A video of the complete system operating on hardware, showing real-time LED brightness control through potentiometer rotation, is available at the following link:

https://drive.google.com/file/d/10-6t50G8t1m8-RIi0KZmDKvxeO3lj-F7/view?usp=drivesdk

IV. Conclusions

This lab assignment successfully demonstrated the design and implementation of an ADC driver architecture for the STM32F411RE microcontroller, fulfilling all three objectives defined in the assignment. The ADC driver was implemented following the hardware abstraction principles established in previous lab assignments, providing a clean and reusable API that isolates direct register operations from the application logic. The Sensor module leveraged this driver to acquire stable analog readings from a potentiometer through a ten-sample averaging algorithm, and the PWM module reused from Lab Assignment 02 translated those readings into proportional LED brightness in real time.

The successful integration of the ADC and PWM modules confirmed the effectiveness of the layered driver architecture across multiple lab assignments. By reusing the GPIO Driver, TIM Driver, and PWM module without modification, the team demonstrated that well-designed hardware abstraction layers are directly portable between applications — a fundamental principle of professional embedded software development.

From a technical standpoint, the lab provided hands-on experience with several key STM32 ADC peripheral concepts. The configuration of the CR1 and CR2 control registers for resolution and trigger mode, the use of SQR3 to select the input channel, the configuration of sampling time through SMPR2, and the polling of the EOC flag in SR to detect conversion completion were all applied and verified against physical hardware behavior.

The challenge encountered during integration — where the circuit did not respond despite successful compilation — reinforced an important lesson in embedded systems development: a correct build does not guarantee correct hardware behavior. Systematic hardware and software verification, layer by layer, is essential to identify issues that are invisible to the compiler. In this case, the missing pwm_start() call was only identified by reviewing the initialization sequence against the expected hardware behavior, highlighting the importance of understanding the complete system flow rather than relying solely on compilation success.

Link GitHub:

Sebastian:
https://github.com/sebastianlizalde/MSE-Lab/tree/main

Michelle:
https://github.com/michelle987654/MSE-Lab/tree/main

Melanie:

https://github.com/melanie-hdez/MSE-Lab.git

References

[1] D. V. Lindberg and H. K. H. Lee, “Optimization under constraints by applying an asymmetric entropy measure,” *J. Comput. Graph. Statist.*, vol. 24, no. 2, pp. 379–393, Jun. 2015, doi: 10.1080/10618600.2014.901225.

[2] B. Rieder, *Engines of Order: A Mechanology of Algorithmic Techniques*. Amsterdam, Netherlands: Amsterdam Univ. Press, 2020.

[3] I. Boglaev, “A numerical method for solving nonlinear integro-differential equations of Fredholm type,” *J. Comput. Math.*, vol. 34, no. 3, pp. 262–284, May 2016, doi: 10.4208/jcm.1512-m2015-0241.